

CockroachDB

Day 3: Production Operations & Security

 **Distributed SQL** — observe it, size it, survive it

Observability · Capacity Planning · Benchmarking · DR · Security



Day 3 Schedule

Module 3: Production Operations & Security

1. Observability Stack — Prometheus, Grafana, log channels



Lab 9 — Build the observability stack with SLO dashboards *(75 min)*

2. Capacity Planning & Hardware Sizing

3. Workload Simulation & Benchmarking



Lab 10 — TPC-C benchmark & capacity sizing *(75 min)*

4. Backup, Restore & Disaster Recovery



Lab 11 — BACKUP/RESTORE/schedules & cross-cluster DR drill *(75 min)*

5. Security Hardening & On-Call Practices



Lab 12 — Security end-to-end: TLS rotation, SSO, audit pipeline *(75 min)*



Where Days 1–2 Left Us

- Day 1: the architecture, and why the *primary key is a distribution decision*
- Day 2: transactions, plan reading, multi-region — and Lab 8's measured throughput numbers

Today changes the question. Days 1–2 asked *"is this schema correct and fast?"* Day 3 asks *"how do I know it still is at 3am, and what do I buy to keep it that way?"*

Every schema decision you made yesterday shows up today as a metric, a capacity number, or a page.



Section 1: The Observability Stack

What to measure, and what to page on

Two Thousand Metrics, Fifteen That Matter

```
curl -s http://localhost:8080/_status/vars | wc -l  
# ~2000
```

A dashboard with 200 panels is a dashboard nobody reads. The job is not to display everything — it's to choose the handful that answer *"is it healthy?"* and *"what changed?"*

CockroachDB exposes metrics three ways:

- `/_status/vars` — Prometheus text format, per node
- `crdb_internal.kv_node_status` — the same numbers via SQL
- DB Console → Metrics — prebuilt dashboards over the same data

The Short List

Metric	Signal	Page when
<code>ranges_unavailable</code>	Lost quorum — data unreadable	> 0 for 1 min 
<code>liveness_livenodes</code>	Nodes the cluster believes are up	< expected 
<code>ranges_underreplicated</code>	Below target replica count	> 0 for 15 min 
<code>sql_service_latency_bucket</code>	p50 / p99 SQL latency	p99 over SLO 
<code>txn_restarts_serializable</code>	40001 retries — contention	rising trend 
<code>sys_cpu_combined_percent_normalized</code>	Per-node CPU	> 0.80 sustained 
<code>capacity_available</code>	Disk headroom	< 20% free 
<code>admission_wait_durations_kv_sum</code>	Admission control queueing (one series per queue)	rising = overload 
<code>rocksdb_read_amplification</code>	Storage engine read cost	> 20 sustained 
<code>changefeed_max_behind_nanos</code>	CDC lag	over freshness SLO 

The one people miss: `txn_restarts_serializable`. A rising restart rate is almost always a *schema* problem surfacing as an ops problem — a single-row counter, a sequential key, an over-wide transaction.

From Metric to SLI to Alert

Recording rules — compute once, use everywhere

```
- record: crdb:sql_latency:p99
  expr: histogram_quantile(0.99, sum(rate(sql_service_latency_bucket[1m])) by (le))

- record: crdb:sql_success_ratio
  expr: |
    1 - ( sum(rate(sql_failure_count[5m]))
      / clamp_min(sum(rate(sql_query_count[5m])), 1) )
```

SLO: 99.9% of statements complete under 100 ms over 30 days.
Error budget: $0.1\% \times 30 \text{ days} \approx 43 \text{ minutes of "too slow"}$.

Burn-Rate Alerting — Page on Trajectory, Not on Blips

```
- alert: CRDBLatencySLOFastBurn
  expr: |
    (1 - ( sum(rate(sql_service_latency_bucket{le="0.1"}[1h]))
      / clamp_min(sum(rate(sql_service_latency_count[1h])), 1) ))
    > (14.4 * 0.001)
  for: 5m
  labels: {severity: critical}
```

Burn rate	Window	Budget gone in	Action
14.4×	1 hour	~2 days	Page immediately
6×	6 hours	~5 days	Page
3×	1 day	~10 days	Ticket
1×	3 days	30 days	Nothing — that's the budget

A threshold alert on "p99 > 100 ms" pages you for a 30-second blip. A burn-rate alert pages you when the month is actually at risk.

Structured Log Channels

CockroachDB routes events to named channels. In production each one goes somewhere different.

Channel	Carries	Ships to
HEALTH	Node liveness, restarts, disk problems	Ops alerting
OPS	Decommissioning, rebalancing, node events	Ops log
SENSITIVE_ACCESS	Audited table reads/writes	SIEM
USER_ADMIN	User/role creation and deletion	SIEM
PRIVILEGES	GRANT / REVOKE	SIEM
SQL_EXEC	Statement execution (verbose)	Debug only
SQL_PERF	Slow queries	Performance triage

```
sinks:
  file-groups:
    security:
      channels: [SENSITIVE_ACCESS, USER_ADMIN, PRIVILEGES]
      auditable: true
      buffered-writes: false # audit records must survive a crash
```



Lab 9: Prometheus + Grafana with SLO Dashboards



75 min

Objectives

- Scrape all three nodes and confirm the targets are healthy
- Write recording rules for p50/p99 and a success-ratio SLI
- Alert on node loss, unavailable ranges, and error-budget burn rate
- Split log channels into health / ops / security sinks
- Capture and redact a `debug zip`



Setup

```
scripts/crdb up # 3 nodes, fixed names for the scrape config
scripts/crdb run workload init kv --drop 'postgresql://root@crdb1:26257?sslmode=disable'
```

Lab Activities

- A: The metrics endpoint and the fifteen that matter
- B: Prometheus with recording and burn-rate rules
- C: Grafana — official dashboards plus a 4-panel SLO view
- D: Log channels split to separate sinks
- E: `debug zip`, redaction, statement bundles



Section 2: Capacity Planning & Hardware Sizing

From a throughput target to a purchase order

The Per-Node Budget

Resource	Guidance	Why
CPU	4–16 vCPU; ≥ 8 for production	Below 4, background work starves foreground SQL
RAM	4 GB per vCPU	Cache 25% + SQL memory 25% + Go heap + page cache
Disk	≤ 2.5 TB per node	Recovery and rebalance time scale with store size
IOPS	≥ 500 IOPS, ≥ 30 MB/s per vCPU	Raft log + LSM compaction are write-amplifying
Network	≥ 1 Gb/s; multi-region: latency $\gg$ bandwidth	Every write is a Raft round trip
Store/CPU	~150 GB/vCPU write-heavy ~320 GB/vCPU read-heavy	Compaction cost scales with data per core

Always set `--cache=.25` `--max-sql-memory=.25`. The defaults are laptop-safe, not production-correct.
A node should be told it owns the machine.

The Number That Decides Everything: Write Amplification

One logical `INSERT` into a table with two secondary indexes, at replication factor 3:

```
1 row × 3 KV writes (base + 2 indexes) × 3 replicas = 9 physical writes
```

Read that again. Adding one index to a hot table increases your write hardware requirement by ~33%. That is a *schema* decision with a *hardware* price tag.

This is why Lab 8's "index tax" measurement matters more than any sizing spreadsheet: the spreadsheet takes the index count as an input.

The Sizing Formula

```
nodes_needed = ceil( target_QPS  
                    / (measured_ops_per_node × utilization_target) )  
  
utilization_target = 0.5
```

Why 0.5 and not 0.8? The cluster must still serve traffic during:

- A node failure (its load redistributes to the survivors)
- A rebalance (background replica movement competes for IO)
- A rolling upgrade (one node down at a time, by design)

Then check the other three constraints — storage capacity, IOPS, and store-per-vCPU — and take the largest node count of the four. The binding constraint is rarely CPU.

Admission Control — Graceful Degradation

When demand exceeds capacity, something has to wait. Admission control chooses *what*.

```
SHOW CLUSTER SETTING admission.kv.enabled;           -- true by default
SHOW CLUSTER SETTING admission.sql_kv_response.enabled;

-- Tag a workload so its work is deprioritized
SET default_transaction_quality_of_service = 'background'; -- analytics
SET default_transaction_quality_of_service = 'critical';   -- OLTP
```

Without admission control, an analytical scan and your checkout path compete as equals and both get slow. With it, the scan queues and checkout keeps its latency. Overload becomes a slowdown of the right thing instead of a collapse of everything.

Benchmarking: Pick the Right Workload

Workload	Shape	Answers
<code>kv</code>	Single-table point ops	Raw per-node ceiling
<code>ycsb</code>	Configurable read/write mix	Read/write sensitivity
<code>tpcc</code>	9 tables, joins, txn mix	"Can this run my OLTP app?"
<code>movr</code>	Multi-region ride-share	Multi-region latency
<code>bank</code>	Transfers between accounts	Contention and retry behaviour
<code>tpch</code>	Read-only analytical queries	Scan-heavy / OLAP-style load

The trap: `kv` numbers look glorious and mean almost nothing for an app with joins, indexes, and foreign keys. Use `kv` for a ceiling; use `tpcc` for what you'll actually get.

Reading TPC-C Output

```
_elapsed_   _tpmC_   _efc_ _avg (ms) _p50 (ms) _p99 (ms) _pMax (ms)
180.0s      126.3    98.2%  12.4    11.0     41.9     71.3
```

- tpmC — new-order transactions per minute
- efc — efficiency vs the theoretical 12.86 tpmC per warehouse
- p99 — the number your users actually feel

Passing = efficiency $\geq$ 85%. Below that, the cluster is the bottleneck. Raise warehouses until it drops, then find out *which* resource bound first — CPU, disk, or contention.

TPC-C also teaches schema design by example: every table leads its primary key with the warehouse id. That's the Per-Tenant Co-located PK pattern, and it's why a new-order transaction touches one range instead of nine.



Lab 10: TPC-C Benchmark & Capacity Sizing



75 min

Objectives

- Run TPC-C and read efficiency, tpmC, and tail latency
- Sweep warehouses until efficiency drops, then name the bottleneck
- Measure your per-node ceiling at three read/write mixes
- Watch admission control protect foreground work
- Produce sizing proposals for 1k / 10k / 100k / 1M QPS



Setup

```
scripts/crdb run workload fixtures import tpcc --warehouses=10 \  
'postgresql://root@crdb1:26257?sslmode=disable' \  
scripts/crdb run workload run tpcc --warehouses=10 --ramp=30s --duration=3m \  
'postgresql://root@crdb1:26257?sslmode=disable'
```

Deliverable

Each pair presents one sizing scenario: node count, instance shape, the binding constraint, and the one schema decision that most changes the answer.



Section 3: Backup, Restore & Disaster Recovery

An untested backup is a hypothesis

The Backup Vocabulary

```
-- Full, from a slightly stale snapshot so it doesn't fight live traffic
BACKUP DATABASE bank INTO 's3://backups/bank?AUTH=implicit'
  AS OF SYSTEM TIME '-10s'
  WITH revision_history;

-- Incremental - auto-detects its parent
BACKUP DATABASE bank INTO LATEST IN 's3://backups/bank?AUTH=implicit';

-- Inspect without restoring
SHOW BACKUPS IN 's3://backups/bank?AUTH=implicit';
SHOW BACKUP FROM LATEST IN 's3://backups/bank?AUTH=implicit';
```

AS OF SYSTEM TIME '-10s' on *every* production backup. `revision_history` is what makes any timestamp in the window restorable, not just the backup instants.

Restore Shapes

```
-- Side by side - never straight over the live database during a drill
RESTORE DATABASE bank FROM LATEST IN '...' WITH new_db_name = 'bank_restored';

-- One table into an existing database
RESTORE TABLE bank.public.accounts FROM LATEST IN '...' WITH into_db = 'staging';

-- Point in time
RESTORE DATABASE bank FROM LATEST IN '...' AS OF SYSTEM TIME '2024-06-01 14:02:00';

-- Whole cluster: users, roles, settings too. Needs a fresh cluster.
RESTORE FROM LATEST IN '...';
```

A database backup does not carry users, roles, or grants. A cluster backup does. Guess which one people discover mid-incident.

RTO and RPO — Honest Arithmetic

worst-case RPO = incremental interval + backup duration + detection time
actual RTO = locate + restore + verify + cut over

Cadence	Nominal RPO	Realistic RPO	Cost
Hourly incremental	1 h	~1 h 5 m	Low
15-minute incremental	15 m	~18 m	Medium
5-minute incremental	5 m	~7.7 m	High
Changefeed to object store	seconds	seconds	Highest

Your RTO is a measurement, not a policy. Restore a real dataset, time it, divide by size, extrapolate. The number in the runbook is almost always smaller than the number on the stopwatch.

The DR Drill That Counts

✓ A real drill

- Restores into a *different* cluster
- Verifies row counts *and* a business checksum
- Confirms users and grants came back
- Records a measured RTO
- Ends with an app smoke test

✗ Theatre

- "The backup job succeeded"
- Restoring over the source
- Checking that *a* row exists
- An RTO copied from a vendor slide
- Never testing the runbook on someone else

Swap runbooks with another pair in the lab. If they can't follow yours without asking you a question, it won't work at 3am either.



Lab 11: BACKUP / RESTORE / Schedules & Cross-Cluster DR



75 min

Objectives

- Full, incremental, and scheduled backups with revision history
- Restore a database, a table, and a point in time — side by side
- Run a real cross-cluster DR drill and verify it
- Compute the RPO your cadence actually guarantees
- Write a runbook someone else can follow



Setup

```
# Cluster A (primary) on 2625x, cluster B (standby) on 2635x  
# Backups go to nodelocal:// so the lab runs offline
```

Deliverable

A completed drill record: backup size, backup duration, transfer time, restore time, measured RTO, and a row-count + checksum match between A and B.



Section 4: Security Hardening

Certificates, roles, authentication, audit

TLS Everywhere — and the Rotation Nobody Plans

```
export CRDB_COMPOSE=docker/labs-secure.yml      # Lab 12 runs against the TLS stack
scripts/crdb run cert create-ca      --certs-dir=/certs --ca-key=/certs/ca.key
scripts/crdb run cert create-node    crdbs1 localhost 127.0.0.1 --certs-dir=/certs --ca-key=/certs/ca.key
scripts/crdb run cert create-client  root --certs-dir=/certs --ca-key=/certs/ca.key
```

List every name, every time. A reissue *replaces* the SAN list, it does not extend it. Drop `crdbs1` and the certificate is still perfectly valid — but every client that connects by that name fails verification the moment the node reloads it.

Rotating Without Downtime

Reissue, then signal the process — no restart, connections stay open

```
docker compose -f docker/labs-secure.yml kill -s HUP crdbs1
```

Only if the binary is PID 1. Through a shell-script entrypoint the signal never reaches it: the cert is reissued, `cert list` shows the new expiry, and the server keeps serving the old one. True of any wrapped process.

Cert	Lifetime	Rotate at
CA	5–10 years	50% of life, planned
Node	1 year	3 months before expiry
Client	1 year (shorter for humans)	Quarterly, automated

Rotating the CA — Order Matters

```
1. Append the NEW CA to ca.crt on every node and client    (both trusted)
2. SIGHUP every node
3. Re-issue node and client certs from the new CA, one node at a time
4. Remove the OLD CA from ca.crt
5. SIGHUP every node again
```

Doing steps 3 and 4 in the wrong order locks you out of your own cluster.
Write the procedure down before you need it, and rehearse it on a
throwaway cluster.

RBAC That Survives CI/CD

```
CREATE ROLE ledger_connect;  
GRANT CONNECT ON DATABASE ledger TO ledger_connect;  
GRANT USAGE ON SCHEMA ledger.public TO ledger_connect;  
  
CREATE ROLE ledger_ro; GRANT ledger_connect TO ledger_ro;  
GRANT SELECT ON ALL TABLES IN SCHEMA ledger.public TO ledger_ro;  
ALTER DEFAULT PRIVILEGES IN SCHEMA ledger.public GRANT SELECT ON TABLES TO ledger_ro;  
  
CREATE ROLE ledger_rw; GRANT ledger_ro TO ledger_rw;  
CREATE ROLE ledger_migrator; GRANT ledger_rw TO ledger_migrator;  
GRANT CREATE ON SCHEMA ledger.public TO ledger_migrator;
```

Do this first, or the rest is decorative: a new database's `public` schema grants `CREATE` to the `public` role — every user can create objects in it.

```
REVOKE CREATE ON SCHEMA ledger.public FROM public;
```

ALTER DEFAULT PRIVILEGES is the feature that matters. Without it, every migration that creates a table also needs a matching `GRANT` — and the one someone forgets is the one that pages you.

Authentication: HBA and SSO

```
SET CLUSTER SETTING server.host_based_authentication.configuration = '  
# TYPE  DATABASE  USER          ADDRESS  METHOD  
host    all         app_user      all      cert  
host    all         report_user   all      cert-password  
host    all         all           all      scram-sha-256  
';
```

```
-- DB Console SSO  
SET CLUSTER SETTING server.oidc_authentication.enabled = true;  
SET CLUSTER SETTING server.oidc_authentication.provider_url = 'https://idp.example.com';  
-- SQL client SSO  
SET CLUSTER SETTING server.jwt_authentication.enabled = true;
```

The operational argument for SSO: deprovisioning a human in the IdP removes their database access. Without it, offboarding depends on someone remembering to run `DROP USER`.

The Audit Pipeline

```
ALTER TABLE ledger.public.accounts EXPERIMENTAL_AUDIT SET READ WRITE;
```

Every access produces a structured record on the `SENSITIVE_ACCESS` channel:

Field	The auditor's question
Timestamp	When
User	Who
TableName	What data
Statement	What operation
ApplicationName	Which service
RemoteAddress	From where

Ship it off-box. An attacker with node access can delete local log files. Audit records that already left the machine cannot be un-sent.

On-Call: What to Capture Before You Escalate

debug zip

```
cockroach debug zip out.zip \  
--host=... --redact
```

- Per-node status and logs
- Changed cluster settings
- Range and lease state
- In-flight queries
- Contention events
- Heap and goroutine dumps

Statement bundle

```
EXPLAIN ANALYZE (DEBUG)  
SELECT ...;
```

- Narrower and usually more useful
- Plan, statistics, trace, environment
- Reproducible by support
- Also available in DB Console → Statements

Always **--redact** anything leaving your organization — a raw zip contains query text with literal values.

Compliance Mapping

Control	Framework	Feature
Encryption in transit	SOC 2, HIPAA	TLS node + client certs
Encryption at rest	SOC 2, HIPAA	Enterprise encryption-at-rest
Least privilege	SOC 2, GDPR	Roles + default privileges
Audit trail	SOC 2, HIPAA	<code>SENSITIVE_ACCESS</code> channel
Right to erasure	GDPR	Row deletion + TTL + backup expiry
Data residency	GDPR	<code>REGIONAL BY ROW</code> + zone configs
Recoverability	SOC 2	Scheduled backups + a measured DR drill
Deprovisioning	SOC 2	SSO / OIDC

Notice how many of these are answered by *schema* and *topology* choices, not by a security product.



Lab 12: Security Hardening End-to-End



75 min

Objectives

- Build a secure cluster; prove insecure access fails
- Rotate node and client certs with zero downtime — verified on the wire
- Build a role hierarchy and test it positively *and* negatively
- Configure HBA: certs for services, SCRAM for humans
- Wire an audit pipeline and ship it off-box



The check people skip

```
openssl s_client -connect localhost:26257 </dev/null 2>/dev/null \
| openssl x509 -noout -dates
```

Reissuing a cert on disk isn't rotation. Rotation is what the server actually serves.

Day 3 Wrap-Up

- Observability — fifteen metrics, burn-rate alerts, split log channels
- Capacity — measure a per-node ceiling, size at 50% utilization, find the binding constraint
- Write amplification — indexes × replicas is a hardware cost set by your schema
- DR — an RTO you measured, a runbook someone else can follow
- Security — rotation rehearsed, least privilege that survives migrations, audit shipped off-box

Tomorrow: CDC in depth, the outbox pattern, ORM and retry integration, migrating off PostgreSQL, and running all of it on Kubernetes.